

Description

Definitions:

Application: executable software, in windows a program ended by *.exe

Algorithm: sequence of operations.

1 Program structure

Software program is composed by 2 parts:

- Data
- Code, operations on data.

Example: program execute $((A + B) / 5) * 3$. In this case data: {A, B}

Data and the code are stored in different memory areas:

- The code manipulates data (sometimes through user intervention), so generally data is read/write.
- The code can't be changed, unless updates are done.

How it works

The program contains instruction in a specific language like C++/C, etc....

1.1 Compiler

So, taking the example above:

- The program will have two variables: **A** and **B**. These variables will be stored in program data memory.
- In order to execute the operation, the code needs access memory where variables **A** and **B** – memory address where they are stored. Something like this:

Memory address	Content	Variable
0x10000010	23	A = 23
0x10000014	52	B = 52

Sequence:

- Read memory 0x10000010 content, which is 23 and store it to A, so A = 23
- Read memory 0x10000014 content, which is 52 and store it to B, so B = 52
- Create temporary variable $Tmp = A + B = 23 + 52 = 75$
- Calculate the next instructions:
 - $Tmp = Tmp / 5 = 75 / 5 = 15$
 - $Tmp = Tmp * 3 = 15 * 3 = 45$

The compiler role is to convert the language code into executing the instructions above, so the CPU can execute the instructions as above.

NOTE: example above expects variables A and B are stored as 4 bytes:

A = [0x10000010...0x10000013] = [0 0 0 23]

B = [0x10000014...0x10000017] = [0 0 0 45]

However, 1 byte could store values: [0, 255]. The A and B could use only 1 memory byte instead of the 4, as presented above. In the later case, the memory would become:

Memory address	Content	Variable
0x10000010	23	A = 23
0x10000011	52	B = 52

How many memory bytes are allocated for variables A and B is determined by the programmer and the programming language. Depending what A and B variables represents in real life, A and B could have different representation.

Variable A 4 bytes memory values: [0, ..., 4294967295] and 1 byte memory values: [0, ..., 255].

When A represents a environment/house temperature, max 255 Celsius is good enough. A temperature for melting metals could be 1000 Celsius. However, when A represents distances in galaxy, A will need 4 or more bytes of memory to store the value.

The programmer decides A variable type and the compiler will allocate the required memory space to store it.

1.2 Program Data

Native C/C++ language supports the following types of data:

- Numbers data, essentially integers and floating
- Alphanumeric data, string. Sample "This is my house".

1.2.1 Integer Data

The integer type is the same as defined by math. The differences are related to how they are stored to memory and CPU type:

- Base
- Integer types based on of memory

1.2.1.1 Integer Base

Decimal: $52 = 5 * 10 + 2$

Hex: $52 = 3 * 16 + 4 = 0x34$

Binary: $52 = B\ 0011\ 01\ 00$

Numbers are stored in memory in binary format but programming languages use mostly hex format.

1.2.1.2 Integer types

There are the following integer types:

- Char: 1 byte. Valid values: [0,..., 255] or [-128,..., 127]
- Short: 2 bytes. Valid values: [0, ..., 65535] or [-32,767, ..., 32,768]
- Int: 4 bytes. Valid values: [0, ..., 4294967295] or [2,147,483,647, ..., 2,147,483,648]
- Long: CPU 32 bytes = 8 bytes,

The memory and CPU does not know how to interpret a number. For example show variable value depending on the type.

Memory address	Value	A type	A value
0x10000010	0x82	Unsigned char	130
0x10000010	0x82	Char	-126

1.2.1.3 TestInt Program

Program TestInt tests integer types and defines one function per type of supported int.

All C/C++ programs are started by function main() executing the sequence of testing various integer types. For each integer type there is a function as follows:

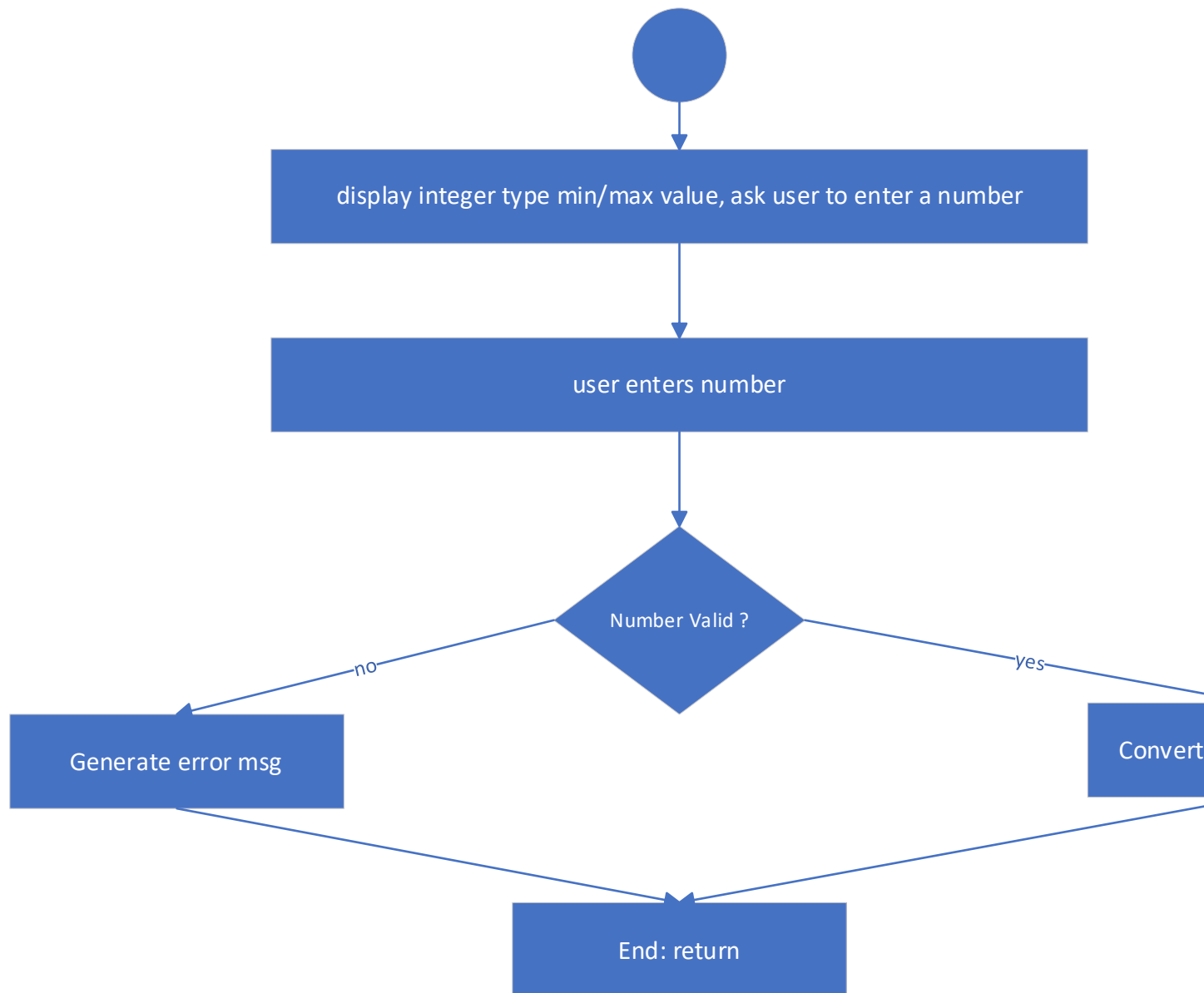
Function	C/C++ type	Definition
processChar()	char	Unsigned, 1 byte
processUChar()	unsigned char	1 byte, [0,..., 255]
processShort()	Short	Unsigned, 2 bytes
.... same rule for other types		

processChar() is analyzed next. The input for each function is the console, defined by C++ language as: **std::in**

The output is the display, defined by C++ language as: **std::out**

The output is the display, defined by C++ language as: **std::err**. In this program is the console.

The input from console is interpreted by computer as a string. So, for example <1234> input is interpreted as string "1234". No arithmetic operations can be executed on strings. Then the program has to convert the string to integer before executing any specific integer operations.



The integer number is displayed in 3 formats: decimal, hex and binary. Binary representation is used sometimes in engineering application but it is not used much

